Se dă un graf neorientat cu *N* fortărețe și *M* drumuri între fortărețe. General Gigel, inginer iscusit, se gândește să bombardeze o singură fortăreață, a.î. să întrerupă comunicarea între cât mai multe fortărețe și tabăra lui să câștige. Descrieți un algoritm pentru găsirea fortăreței.

1p - Găsim toate punctele de articulatie și componentele biconexe din jurul lor. Gigel va bombarda punctul de articulație care are în jurul său cele mai multe componente biconexe.

0.5p - Întoarce orice punct de articulație

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mici numere. Secvența este prea mare pentru a fi stocată integral în memorie.

n log k- folosim un maxheap

n * k - 0.5 din punctaj

Se dă un graf neorientat cu *n* noduri cu costuri pozitive pe muchii și *k* surse (k mult mai mic decât n). Pentru fiecare nod din graf se cere drumul de lungime minima față de cea mai apropiata sursă. Descrieți un algoritm eficient pentru această problemă.

Sol1: Se poate adaugă o sursă fictivă cu muchie de cost 0 către toate sursele inițiale și să se facă Dijsktra din aceeastă sursă.

Sol2: Se adaugă toate sursele în heap și continuă cu Dijkstra. Complexitatea este O(M log N). Se puncteaza cu 0.5p o soluție mai ineficientă.

Cum se poate implementa eficient un algoritm de programare dinamică având recurența:

$$dp[n] = \begin{cases} \mathrm{dp}[n-2] - \mathrm{dp}[n-4], & n \geq 4 \\ 2 * n, & n < 4 \end{cases}$$

Exponențiere logaritmică pe matrice

Tribul A este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *X*.
Tribul B este construit sub forma unui graf unde pentru a te muta dintr-un loc în alt loc către care există muchie te costă *Y*.
Fiecare trib are un (singur) loc de unde se poate lua bacul pentru a traversa râul și a ajunge la celălalt trib cu costul *Z*. Aceasta este singura metodă de a ajunge dintr-o parte în alta.
Ce algoritm ați implementa pentru a găsi drumul cu costul cel mai mic dintr-un loc din A (sursă) până la un loc din B (destinație).

pentru BFS maxim, dacă zic BF pentru că au considerat muchii negative also maxim,

dijktra si bf fara explicatie suplimentara, jumatate

Carrefour Orhideea dorește să afle în câte moduri se pot aranja fructele în tăvițe. Se dau *n* tăvițe și *m* fructe (m <= n). Un fruct poate fi pus în orice tăviță. Descrieți un algoritm pentru această problemă.

Se foloseste backtracking pentru a genera toate modalitatile de aranjare pe tavite care respecta restrictiile din enunt.

Cum ați modifica algoritmul lui Dijkstra pentru a calcula al doilea cel mai scurt drum dintr-un graf?

1p - Se păstrează cele mai mici 2 distanțe posibile până la fiecare nod.

0.5p - se scoate pe rând câte o muchie și se rulează Dijkstra

Descrieți un algoritm care primește la intrare, pe rând, elementele dintr-o secvență de *N* numere și calculează top *K* (K semnificativ mai mic decât N) cele mai mici numere. Secvența este prea mare pentru a fi stocată integral în memorie.

n log k- folosim un maxheap

n * k - 0.5 din punctaj

În orașul X, rețeaua de internet este răspândita în toate cartiere, în fiecare cartier fiind amplasate mai multe routere interconectate între ele. Unele routere cedează, iar internetul pică în unul sau mai multe cartiere. Ce algoritm ați folosi pentru identificarea routerelor care duc la astfel de căderi și ce soluție sugerați pentru scăderea numărului de incidente?

Găsim toate punctele de articulație. O soluție ar fi adăugarea de muchii redundante intre cartiere.

Cum se poate calcula eficient al *K*-lea cel mai mic număr dintr-o secvență de N numere?

1p - n (folosind functia de partitionare de la qsort)

0.5p - n log k

Se dă un graf neorientat cu *n* noduri cu costuri pozitive pe muchii și *k* surse (k mult mai mic decât n). Pentru fiecare nod din graf se cere drumul de lungime minima față de cea mai apropiata sursă. Descrieți un algoritm eficient pentru această problemă.

Sol1: Se poate adaugă o sursă fictivă cu muchie de cost 0 către toate sursele inițiale și să se facă Dijsktra din aceeastă sursă.

Sol2: Se adaugă toate sursele în heap și continuă cu Dijkstra. Complexitatea este O(M log N). Se puncteaza cu 0.5p o soluție mai ineficientă.

Cum puteți **afișa toate** drumurile distincte între 2 noduri dintr-un graf **orientat aciclic**?

Există un număr exponențial de drumuri, deci ar trebui calculate cu backtracking